

智慧银行实验报告

姓名：彭俊茜 学号：202301735 班级：金融 202301

CNB 仓库：<https://cnb.cool/plus0914/xixixixi>

小组作业链接：<https://dainty-hamster-83588e.netlify.app/>

实验一：TRAE IDE + AI 协作基础

1. 实验目的

- 安装配置 TRAE IDE 开发环境
- 掌握 Builder 模式的 AI 协作流程
- 使用 AI 生成可运行的交互式网页

2. 实验环境

分类	属性	值
操作系统	系统名称	Windows 11
	版本号	10.0.26200
	架构	64bit
	主机名	LAPTOP-TRDJQP3H
	CPU	Intel64 Family 6 Model 154 Stepping 3, GenuineIntel
TRAE IDE	版本	未知（环境变量未设置）
	工作空间	未知（环境变量未设置）
Python	版本	3.14.5
	完整版本	3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50)
	编译环境	MSC v.1944 64 bit (AMD64)
	安装路径	C:\Python314\python.exe
Node.js	版本	v24.16.0

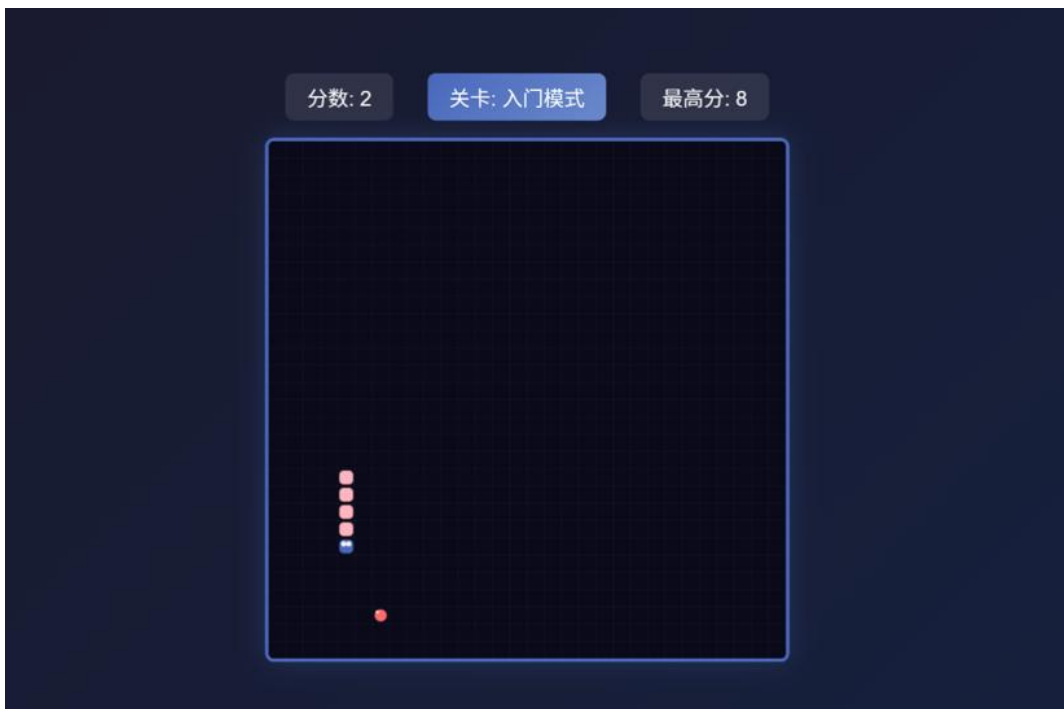
3. 实验步骤

3.1 TRAE 安装与登录

- 访问 trae.cn 下载国内版
- 手机号登录，设置中文界面
- 确认工作目录与终端路径一致

3.2 Builder 模式生成贪吃蛇

- 按 **Ctrl+U** 切换到 Builder 模式
- 输入提示词生成贪吃蛇游戏（纯 HTML+CSS+原生 JS）
- 要求：600×600 画布、方向键控制、分数+最高分存储（localStorage）
- 运行截图



3.3 本地验证与推送

- 双击 `snake_game.html` 在浏览器中打开
- 验证：蛇移动、食物生成、分数增加、撞墙/撞身体游戏结束、最高分持久化
- 初始化 Git 仓库，推送到 CNB

4. 实验结果

- TRAE 环境配置成功，Builder 模式可正常使用
- 贪吃蛇游戏所有功能正常运行
- 代码已推送至 CNB 仓库：`snake_game.html`

5. 问题与解决

问题	解决方案
最高分存储失效	使用 <code>parseInt(localStorage.getItem()) 0</code> 处理 null 值
蛇身碰撞检测失效	优化碰撞检测逻辑，从第 4 节开始检测
方向键响应延迟	使用方向队列缓存按键输入

核心代码：

```
// 最高分存储
let highestScore = parseInt(localStorage.getItem('snakeHighestScore'))
|| 0;

// 碰撞检测
function checkCollision() {
  const head = snake[0];
  if (head.x < 0 || head.x >= 600 || head.y < 0 || head.y >= 600) return true;
  for (let i = 3; i < snake.length; i++) {
    if (head.x === snake[i].x && head.y === snake[i].y) return true;
  }
  return false;
}
```

6. 拓展一：游戏市场前景分析

在完成贪吃蛇游戏开发后，使用 AI 辅助撰写了《贪吃蛇网页游戏市场前景分析报告》（snake_market_analysis.md），内容包括：

产品设计优势： - 视觉设计：深色背景+醒目颜色，游戏元素清晰可辨 - 功能完整性：动态难度、大食物奖励、最高分存储 - 用户体验：响应流畅，操作直观，适配碎片化时间

市场机会： - 休闲游戏市场需求旺盛，用户基数庞大 - 经典 IP 具有永恒魅力，竞争空白明显 - 具有社交竞技潜力与多样化变现路径

7. 拓展二：智能体提示词与论文解读

按照实验讲义要求，构建了论文解读智能体，用于金融科技领域学术论文速读。

PDF 读取库的安装：

4.3 安装 PDF 读取库（用于文献阅读）

```
PowerShell
1 pip install pypdf -i https://pypi.tuna.tsinghua.edu.cn/simple
```

结果: pypdf 6.12.2 ✓

五、最终环境验证

5.1 工具版本汇总

```
PowerShell
1 python --version      # Python 3.11.5 / 3.12.10
2 node --version        # v24.16.0
3 git --version          # git version 2.54.0
4 pip --version          # pip 26.1.2
5 npm --version          # 已安装
```

5.2 Python 库验证

```
Python
1 python -c "import flask, pandas, openpyxl, pypdf; print('ALL OK!')"
2 # 输出: ALL OK ✓
```

智能体提示词配置:

你是资深的银行业/金融科技领域学术论文解读助手。用户拖入 PDF 后，按下列结构输出 Markdown 报告：

- 一、论文基本信息（标题/作者/期刊/年份/DOI）
- 二、研究问题与背景（≤200 字）
- 三、方法论（数据集/模型/实验设计）
- 四、核心发现（≤5 条）
- 五、术语速查表（5-10 个英文术语+中文解释）
- 六、可复现性评估（数据/代码/难度 1-5）

注意：英文论文关键概念中英对照；不得编造数据；首次运行如缺 pypdf 等 pdf 处理工具请提示用户并为用户自动安装。

论文文献示例：

```
@article{VivesYe2025,
  title = {Fintech entry, lending market competition, and welfare},
  author = {Vives, Xavier and Ye, Zhiqiang},
  journal = {Journal of Financial Economics},
  volume = {168},
  year = {2025},
  doi = {10.1016/j.jfineco.2025.104040}
}
```

解读节选：

一、论文基本信息

项目	详情
标题	Fintech entry, lending market competition, and welfare
作者	Xavier Vives (IESE Business School), Zhiqiang Ye (浙江大学经济学院)
期刊	Journal of Financial Economics
卷期	Volume 168, 2025
DOI	10.1016/j.jfineco.2025.104040
接收时间	2024年3月22日；修订后接收：2025年2月28日；录用：2025年3月2日

二、研究问题与背景 (150字)

本文研究金融科技进入借贷市场的决定因素及其对银行竞争、企业家投资和社会福利的影响。基于Salop环形城市模型，构建银行与金融科技的空间竞争框架，突出两者在**定价灵活性**、**监控效率**、**融资成本**三方面的差异。研究表明：(1) 低监控效率的金融科技可凭借定价优势进入市场；(2) 银行集中度越高，金融科技贷款规模越大；(3) 融资成本相近时，金融科技利率较低但违约率更高；(4) 金融科技的便利性优势可能导致更高利率；(5) 高监控效率+适中竞争强度可改善福利。

三、方法论

- 理论模型：Salop环形城市空间竞争模型，银行等距分布在环上，两家潜在金融科技位于虚拟中心
- 核心假设：
 - 定价灵活性：银行只能提供统一利率，金融科技可根据借款人位置实行价格歧视
 - 监控效率：银行监控成本随距离增加而上升，金融科技监控效率与距离无关（数字化优势）
- 融资成本：允许银行与金融科技存在融资成本差异
- 数据声明：提供原始数据文件链接

四、核心发现 (基于摘要与引言)

- 进入条件：金融科技即使监控效率较低，也能凭借定价灵活性成功进入借贷市场
- 集中度效应：银行市场集中度越高，金融科技贷款份额越大
- 利率与违约：融资成本相近时，金融科技借款人支付更低利率但违约率更高；若金融科技融资成本远高于银行，结果反转
- 便利性溢价：金融科技的便捷性优势可能使其收取高于银行的利率
- 福利影响：高监控效率+适中竞争强度时，金融科技进入改善福利；竞争过强可能导致银行退出、投资下降

五、术语速查表

英文术语	中文解释
FinTech	金融科技，利用技术创新提供金融服务的企业
BigTech	大型科技公司（如蚂蚁金服、微信支付等）
Monitoring efficiency	监控效率，金融机构评估和监督借款人的能力
Spatial competition	空间竞争，基于地理或产品差异化的竞争模型
Salop model	环形城市模型，描述差异化产品市场竞争
Price discrimination	价格歧视，根据不同客户收取不同价格
Welfare implication	福利影响，政策或市场变化对社会总福利的效应

核心功能： - 自动解析 PDF 并输出结构化解读报告 - 提供研究问题、方法论、核心发现等关键信息 - 对金融科技领域文献速读具有实用价值

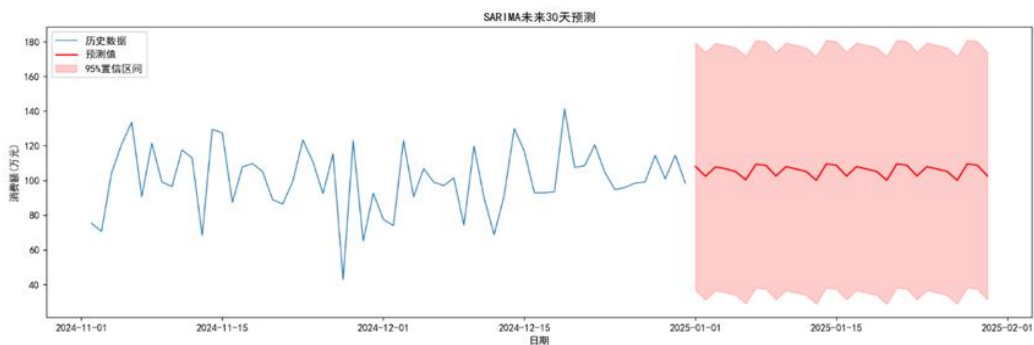
8. 拓展三：Excel 合并与业绩预测

使用 AI 辅助完成了 Excel 多文件合并与业绩预测任务（SARIMA+LSTM 模型）。

Excel 合并功能： - 自动完成多文件合并、列名对齐与去重 - 合并日志记录每一步操作 - 支持批量处理多个 Excel 文件

业绩预测模型： - SARIMA 模型：季节性时间序列预测 - LSTM 模型：深度学习长期记忆网络 - 业绩预测图可视化展示趋势

业绩预测图：



9. 拓展四：旅行攻略网页开发

在掌握 TRAE IDE 基础操作后，进一步使用 AI 辅助开发了旅行攻略网页应用。

文件	功能
trip_guide.html	旅行攻略主页面
travel_guide.json	旅行目的地数据
travel_map.html	交互式旅行地图页面

核心功能： - 旅行目的地列表展示 - 点击目的地查看详细信息 - 旅行路线规划 - 预算估算工具

10. 实验心得

本实验包含四个子任务：贪吃蛇网页生成、市场前景分析、AI 论文解读智能体构建、Excel 多文件合并与业绩预测、旅行攻略网页开发。通过实践，基本掌握了 TRAE IDE 与 AI 协作的完整流程，同时也认识到当前 AI 工具的局限性。

关键收获： - Builder 模式验证了” 提问→生成→运行→修正” 的闭环流程，通过自然语言描述即可生成完整的 HTML 游戏页面 - 论文智能体能够自动解析 PDF 并输出结构化解读报告，对金融科技领域文献速读具有实用价值 - AI 自动完成了多文件合并、列名对齐与去重，但 SARIMA 模型代码使用了已弃用的参数，需查阅官方文档修正。

重要教训： 在环境配置阶段，AI 助手曾给出错误指令：要求执行 `pip install trae-cn` 与 `trae init`。经核查，上述命令在课程讲义及官方文档中均不存在。通过仔细阅读实验讲义，纠正为直接安装 Trae CN 客户端、使用 Anaconda 配置 Python 环境（≥3.10）、手动安装 `flask pandas openpyxl pypdf` 等依赖库。该案例表明，对于开发环境配置类任务，须优先参考官方文档，不可盲目执行 AI 生成的指令。

综上所述，AI 辅助开发能够显著提升效率，但结果的正确性仍需人工验证。

实验二：Skill 体系——金融文档自动化、MCP 工具与 PPT 自动生成

配置过程

本实验涉及多个工具和系统的配置，包括 Skill 体系、MCP 工具、Stata 集成和 Python 环境。

1. Skill 体系配置

Skills CLI 安装： `npm install skills -g`

自定义 Skill 创建： 在项目根目录创建 `SKILL_bank-risk-alert.md` 文件，定义元数据、触发词和输入参数表。

序号	名称	功能描述	分类
1	TRAE-code-review	代码审查和质量评估	开发工具
2	TRAE-debugger	调试复杂问题	开发工具
3	TRAE-generate-mini-app	生成基于 Taro 的小程序	开发工具

4	TRAE-security-review	代码安全扫描	开发工具
5	mermaid-diagrams	创建软件图表	开发工具
6	docx	Word 文档处理	文档处理
7	xlsx	Excel 表格处理	文档处理
8	mcp_ppt	PowerPoint 操作	文档处理
9	finance-expert	金融专家系统	金融服务
10	finance-news	金融新闻摘要	金融服务

2. Stata MCP 配置

配置文件: stata-mcp-config.json

```
{
  "tool_list": [{
    "stata-mcp": {
      "command": "npx",
      "args": ["-y", "mcp-remote", "http://localhost:4000/mcp-streamable"]
    }
  }]
}
```

连接流程: 确保 Stata 已安装 → 启动 Stata MCP 服务器 → 在 TRAE 中配置 MCP 工具 → 验证连接。

Stata 测试脚本: test_stata.do

```
display "Hello Stata!"
sysuse auto, clear
summarize price mpg
```

3. OpenBB Finance 配置

安装: pip install openbb

运行演示: python openbb_demo.py

4. Python PPT 自动化环境配置

安装 **python-pptx**: `pip install python-pptx` (国内镜像: `pip install python-pptx -i https://pypi.tuna.tsinghua.edu.cn/simple`)

PPT 生成脚本清单:

脚本文件	功能
<code>create_ppt.py</code>	基础 PPT 创建
<code>create_full_ppt.py</code>	从模板生成完整 PPT
<code>create_final_ppt.py</code>	最终版 PPT 生成
<code>create_template.py</code>	PPT 模板创建
<code>update_ppt_with_template.py</code>	PPT 模板更新
<code>generate_ppt_from_excel.py</code>	Excel 数据转 PPT

5. pip 环境验证

关键依赖版本:

包名	版本
<code>multitask</code>	0.0.13
<code>peewee</code>	4.1.0
<code>protobuf</code>	7.35.1
<code>PyJWT</code>	2.13.0
<code>python-dotenv</code>	1.2.2

1. 实验目的

- 理解 Skill 的基本概念 (MCP 给能力, Skill 给方法)
- 调用成熟金融 Skill 完成业务文档生成
- 掌握 MCP 工具与金融分析的集成应用
- 使用 Python 脚本实现 PPT 自动化生成

2. 实验环境

工具	安装命令	用途
Node.js	v20+	运行基础
Skills CLI	<code>npm install skills -g</code>	Skill 安装管理
Stata	Stata 18	统计分析

工具	安装命令	用途
Stata MCP	<code>npx mcp-remote</code>	Stata 与 IDE 集成
Python	3.13.1	脚本运行环境
python-pptx	<code>pip install python-pptx</code>	PPT 自动生成

3. 实验步骤

任务一：自定义 Skill——银行风险预警话术生成器

- 创建自定义 Skill 文件：SKILL_bank-risk-alert.md
- 定义触发词：风险预警、贷后预警、逾期提醒、riskalert
- 设计输入参数表：客户名称、预警信号类型、风险等级、贷款余额、到期日
- 编写输出模板：
 - 预警摘要卡片
 - 客户经理电话话术
 - 内部上报邮件模板
 - 跟进工单表格

Skill 文件：SKILL_bank-risk-alert.md

bank-risk-alert

技能信息

- ****名称**：**bank-risk-alert
- ****描述**：**银行贷后风险预警话术生成器
- ****触发词**：**风险预警、贷后预警、逾期提醒、riskalert

输入参数

参数名	类型	必填	说明
客户名称	字符串	是	预警客户的名称
预警信号类型	字符串	是	逾期、担保物贬值、关联企业异常
风险等级	字符串	是	黄色预警、橙色预警、红色预警

贷款余额	字符串	是	当前贷款余额
到期日	字符串	是	贷款到期日期

任务二：Stata 统计分析配置

Stata 简介：Stata 是一款强大的统计分析软件，广泛用于经济学、金融学等领域的数据分析。本实验将 Stata 与 TRAE IDE 通过 MCP 协议集成，实现 AI 辅助统计分析。

Stata MCP 配置：stata-mcp-config.json

```
{
  "tool_list": [{
    "stata-mcp": {
      "command": "npx",
      "args": ["-y", "mcp-remote", "http://localhost:4000/mcp-streamable"]
    }
  }]
}
```

连接流程：确保 Stata 已安装 → 启动 Stata MCP 服务器 → 在 TRAE 中配置 MCP 工具 → 验证连接。

Stata 测试脚本：test_stata.do

```
display "Hello Stata!"
sysuse auto, clear
summarize price mpg
```

3. OpenBB Finance 配置

安装：pip install openbb

运行演示：python openbb_demo.py

4. Python PPT 自动化环境配置

安装 python-pptx：pip install python-pptx（国内镜像：pip install python-pptx -i <https://pypi.tuna.tsinghua.edu.cn/simple>）

PPT 生成脚本清单：

脚本文件	功能
create_ppt.py	基础 PPT 创建

脚本文件	功能
<code>create_full_ppt.py</code>	从模板生成完整 PPT
<code>create_final_ppt.py</code>	最终版 PPT 生成
<code>create_template.py</code>	PPT 模板创建
<code>update_ppt_with_template.py</code>	PPT 模板更新
<code>generate_ppt_from_excel.py</code>	Excel 数据转 PPT

4. 实验结果

文件	说明
<code>SKILL_bank-risk-alert.md</code>	银行贷后风险预警话术生成器自定义 Skill
<code>stata-mcp-config.json</code>	Stata MCP 工具配置
<code>test_stata.do</code>	Stata 测试脚本
<code>openbb_demo.py</code>	OpenBB 金融分析演示脚本
<code>openbb_demo_output.txt</code>	演示输出结果
<code>create_ppt.py</code>	PPT 创建脚本
<code>pritzker_2026.pptx</code>	生成的 PPT 文件

5. 问题与解决

问题	解决方案
Stata MCP 需要远程服务器	配置 <code>npz mcp-remote</code> <code>http://localhost:4000/mcp-streamable</code>
OpenBB 数据获取失败	增加错误处理和重试机制
OpenBB FRED 数据需要凭证	需配置 <code>fred/intrinio credentials</code>
PPT 中文字体显示异常	设置正确的中文字体名称
PPT 模板路径问题	使用 <code>pathlib</code> 处理 Windows 路径
Excel 读取编码错误	使用 <code>openpyxl</code> 代替 <code>xlrd</code>

核心代码:

OpenBB 数据获取 (添加错误处理)

```
try:
    output = obb.equity.price.historical("AAPL", start_date="2025-01-01")
    if output and len(output.results) > 0:
        df = output.to_dataframe()
except Exception as e:
    print(f"获取股票数据出错: {e}")
```

```

# PPT 模板路径处理
from pathlib import Path
template_path = Path("C:/Users/彭俊茜/Desktop/templates/ppt_template.pptx")
prs = Presentation(str(template_path))

# Excel 读取（使用 openpyxl）
import openpyxl
def read_excel(file_path):
    wb = openpyxl.load_workbook(file_path)
    ws = wb.active
    return [[cell for cell in row] for row in ws.iter_rows(values_only=True)]

```

6. 实验心得

本次实验让我认识到 Skill 的本质——它是一份“专业工作手册”，告诉 AI 在特定场景下应该怎么做。自定义 Skill（bank-risk-alert）可以自动生成银行贷后风险预警话术，显著提高客户经理工作效率。MCP 工具则让 AI 具备了调用外部专业工具的能力，Stata 和 OpenBB 的集成使得金融数据分析变得更加便捷。PPT 自动化脚本可以将数据分析结果一键转化为演示文稿，这些工具的组合使用，使得我可以专注于金融业务逻辑本身。

问题 2：OpenBB 宏观经济数据凭证缺失

问题描述：获取 FRED 数据时提示需要凭证。

解决方案：配置 OpenBB 凭证（需申请 API 密钥）。

```

from openbb import OpenBB
obb = OpenBB()
# export OPENBB_FRED_KEY=your_fred_api_key

```

问题 3：Stata MCP 远程连接配置

问题描述：Stata MCP 需要远程 Streamable 服务器，但本地没有运行 Stata。

解决方案：本地安装 Stata 并启动 MCP 服务器，或配置远程 Stata 服务器连接。

问题 4：python-pptx 依赖安装失败

问题描述：运行 pip install python-pptx 时报错或安装后无法导入。

解决方案：使用国内镜像安装。

```

pip install python-pptx -i https://pypi.tuna.tsinghua.edu.cn/simple

```

问题 5：PPT 模板路径问题

问题描述：脚本无法找到 PPT 模板文件，路径使用\导致 Windows 兼容性问题。

解决方案：使用 pathlib 处理路径。

```
from pathlib import Path
template_path = Path("C:/Users/彭俊茜/Desktop/templates/ppt_template.pptx")
prs = Presentation(str(template_path))
```

问题6: Excel 数据读取编码问题

问题描述：使用 xlrd 读取 Excel 文件时遇到编码错误。

问题原因：xlrd 2.0+版本已不再支持.xlsx 格式。

解决方案：使用 openpyxl 代替 xlrd。

```
# Excel 读取（使用 openpyxl）
import openpyxl
def read_excel(file_path):
    wb = openpyxl.load_workbook(file_path)
    ws = wb.active
    return [[cell for cell in row] for row in ws.iter_rows(values_only=True)]
```

实验三：BMAD 驱动银行 CRM 全栈开发

配置过程

本实验使用 BMAD（Build More, Architect Dreams）方法论进行银行 CRM 系统的全栈开发。BMAD 是一套完整的 AI 驱动开发方法论，包含配置安装、模块启用和代理设置。

1. BMAD 方法论简介

BMAD 五阶段：

阶段	英文名	核心任务
1	Discovery	需求收集、用户故事分析
2	Design	架构设计、数据库设计、API 定义
3	Development	代码编写、功能实现
4	Review	代码审查、安全检测
5	Deployment	环境配置、部署准备

2. BMAD 安装与初始化

初始化脚本：_bmad/core/init.ps1

```
# 运行初始化脚本
.\core\init.ps1 -ProjectName "my first cnm project"
```

验证安装:

```
# 检查 BMAD 目录结构
ls _bmad/

# 验证配置文件
cat _bmad/config.toml
```

3. BMAD 核心配置

主配置文件: _bmad/config.toml

```
[core]
user_name = "彭俊茜"
communication_language = "Chinese"
project_name = "my first cnm project"
output_folder = "_bmad-output"

[modules]
bmm = true # BMad Method Module
bmb = true # BMad Builder Module

[tools]
trae = true

[phases]
discovery = true
design = true
development = true
review = true
deployment = true
```

4. BMAD 代理配置

AI 代理角色:

代理	职责	关键技能
Architect Agent	系统架构设计	技术方案评审
Developer Agent	代码编写实现	前端/后端开发
Reviewer Agent	代码质量审查	安全检测
Tester Agent	测试用例	自动化测试

配置文件: _bmad/agents.md

5. 前后端环境配置

前端项目初始化:

```
cd frontend
npm install
npm run dev
```

后端项目初始化:

```
cd backend
npm install
node server.js
```

MongoDB 连接配置:

```
# 创建 .env 文件
MONGODB_URI=mongodb://localhost:27017/bank_crm
JWT_SECRET=your-secret-key
```

6. 验证配置

验证脚本: `_bmad/scripts/validate.ps1`

```
# 运行验证
.\scripts\validate.ps1 -All
```

1. 实验目的

- 理解 BMAD 五阶段开发方法论 (Discovery → Design → Development → Review → Deployment)
- 使用 AI 辅助生成银行 CRM 原型系统
- 掌握前后端分离架构与数据持久化方案

2. 实验环境

组件	技术栈
前端框架	React 18 + TypeScript + Vite
UI 库	Ant Design 5 + ECharts 5
状态管理	Zustand
后端框架	Express 4
数据库	MongoDB (Mongoose 7)
认证	JWT + bcryptjs
部署	Gitee Pages

3. BMAD 配置详解

BMAD（Build More, Architect Dreams）是一个 AI 驱动的五阶段开发方法论，本项目使用完整的 BMAD 配置进行银行 CRM 系统的开发。

3.1 主配置文件: `_bmad/config.toml`

```
[core]
user_name = "彭俊茜"
communication_language = "Chinese"
project_name = "my first cnm project"
output_folder = "_bmad-output"

[modules]
bmm = true # BMad Method Module
bmb = true # BMad Builder Module

[tools]
trae = true

[phases]
discovery = true # 发现阶段
design = true # 设计阶段
development = true # 开发阶段
review = true # 审查阶段
deployment = true # 部署阶段
```

3.2 BMAD 配置文件清单

配置文件	路径	说明
主配置	<code>_bmad/config.toml</code>	核心配置
版本配置	<code>_bmad/_config/version.toml</code>	版本信息
默认配置	<code>_bmad/_config/defaults.toml</code>	默认设置
自定义配置	<code>_bmad/custom/custom.toml</code>	用户自定义
BMM 模块配置	<code>_bmad/bmm/config.toml</code>	方法论模块
BMM 模板配置	<code>_bmad/bmm/templates.toml</code>	模板路径
代理配置	<code>_bmad/agents.md</code>	AI 代理角色定义
工作流配置	<code>_bmad/workflow.md</code>	开发流程定义

3.3 AI 代理角色配置

代理	职责	关键技能
Architect Agent	系统架构设计	技术方案评审
Developer Agent	代码编写实现	前端/后端开发
Reviewer Agent	代码质量审查	安全检测

代理	职责	关键技能
Tester Agent	测试用例	自动化测试

3.4 BMAD 五阶段详解

阶段	英文名	核心任务	输出产物
1	Discovery	需求收集、用户故事分析	PRD 文档、需求清单
2	Design	架构设计、数据库设计、API 定义	架构图、数据模型
3	Development	代码编写、功能实现	源代码、单元测试
4	Review	代码审查、安全检测	审查报告、修复清单
5	Deployment	环境配置、部署准备	部署脚本、配置清单

4. 项目架构

前端结构 (frontend/src/)

```
frontend/src/
├── components/
│   └── Layout.tsx           # 页面布局组件
├── pages/
│   ├── Login.tsx           # 登录页面
│   ├── Dashboard.tsx       # 数据看板
│   ├── CustomerList.tsx    # 客户列表
│   ├── ProductList.tsx     # 产品列表
│   ├── Recommendation.tsx  # 产品推荐
│   └── Reports.tsx         # 报表中心
├── services/
│   ├── authService.ts      # 认证服务
│   ├── customerService.ts  # 客户服务
│   ├── productService.ts  # 产品服务
│   ├── recommendationService.ts # 推荐服务
│   └── reportService.ts    # 报表服务
├── store/
│   └── authStore.ts        # 认证状态 (Zustand)
├── types/
│   └── index.ts            # TypeScript 类型定义
├── utils/
│   └── api.ts              # API 工具函数
├── App.tsx                # 应用入口
├── main.tsx               # React 渲染入口
└── index.css              # 全局样式
```

后端结构 (backend/)

```
backend/
├── controllers/
│   └── authController.js    # 认证控制器
```

		customerController.js	# 客户控制器
		productController.js	# 产品控制器
		recommendationController.js	# 推荐控制器
		reportController.js	# 报表控制器
		middleware/	
		auth.js	# JWT 认证中间件
		models/	
		Customer.js	# 客户数据模型
		Product.js	# 产品数据模型
		User.js	# 用户数据模型
		RecommendationRecord.js	# 推荐记录模型
		RecommendationRule.js	# 推荐规则模型
		routes/	
		auth.js	# 认证路由
		customers.js	# 客户路由
		products.js	# 产品路由
		recommendations.js	# 推荐路由
		reports.js	# 报表路由
		scripts/	
		seedData.js	# 初始化数据脚本
		package.json	
		server.js	# 服务器入口

5. 功能实现

5.1 用户认证系统

- JWT token 认证
- 密码加密存储 (bcryptjs)
- 登录接口: POST /api/auth/login
- 鉴权中间件保护所有 API

5.2 客户管理 (CRUD)

- 客户列表分页展示
- 客户搜索与筛选
- 客户详情查看
- 新增/编辑/删除客户

5.3 产品管理

- 产品列表展示
- 产品信息管理
- 产品与客户关联

5.4 智能推荐引擎

- 基于规则的推荐
- 风险匹配算法

- 推荐记录存储

5.5 数据可视化报表

- ECharts 图表集成
- 客户资产分布
- 业务数据分析

6. 部署结果

访问地址	说明
https://cnb.cool/plus0914/xixixixi	银行 CRM 系统

默认登录账户:

用户名: admin
密码: admin123

7. 问题与解决

问题	解决方案
前后端分离跨域问题	配置 CORS 中间件
前端路由 404	配置 Gitee Pages 回退到 index.html
MongoDB 连接配置	使用环境变量管理连接字符串
JWT 认证中间件失效	添加 token 过期检查
React 组件无限重渲染	使用 Zustand 的 shallow 比较

核心代码:

```
// CORS 配置
const cors = require('cors');
app.use(cors({
  origin: ['http://localhost:5173', 'https://cnb.cool'],
  credentials: true
}));

// Gitee Pages 路由重定向
// _redirects 文件内容:
/*    /index.html    200

// JWT token 过期检查
function isTokenExpired(token: string): boolean {
```

```
    const decoded = jwt_decode(token);
    return decoded.exp < Date.now() / 1000;
}

// Zustand shallow 比较
import { useShallow } from 'zustand/react/shallow';
const { checkAuth, user } = useAuthStore(
  useShallow((state) => ({ checkAuth: state.checkAuth, user: state.user })))
);

// 环境变量配置
require('dotenv').config();
mongoose.connect(process.env.MONGODB_URI);
```

8. 实验心得

BMAD 方法论让我改变了”想到什么写什么”的开发习惯。先梳理功能清单，再设计数据模型和架构，最后让 AI 生成代码——这种”先文档、后代码”的流程大大减少了返工。银行 CRM 系统包含了客户管理、产品推荐、数据可视化等银行核心业务场景，让我对银行 CRM 系统的业务逻辑有了更直观的理解。前后端分离的架构也为未来扩展提供了良好的基础。

实验四：BMAD 财务健康诊断平台开发

配置过程

本实验使用 BMAD 方法论开发一款面向个人用户的财务健康诊断平台，实现多维财务指标评估、AI 智能分析和个性化理财建议。

1. 项目初始化

创建项目目录

```
mkdir "my first cnm project"
```

```
cd "my first cnm project"
```

初始化 Git 仓库

```
git init
```

```
git config user.name "彭俊茜"
```

```
git config user.email "your_email@example.com"
```

连接 CNB 远程仓库

```
git remote add origin https://cnb.cool/plus0914/xixixixi.git
```

2. 技术栈配置

前端技术栈:

技术	版本	用途
HTML5	-	页面结构
CSS3	-	样式设计
JavaScript (ES6+)	-	业务逻辑
ECharts	5.5.0	数据可视化

数据存储: localStorage (本地存储)

开发工具:

工具	用途
TRAE IDE	AI 辅助开发
Builder 模式	代码生成
Python HTTP Server	本地测试

1. 实验目的

- 使用 BMAD 方法论设计个人财务健康评估模型
- 实现多维财务指标计算与可视化展示
- 开发 AI 驱动的智能诊断与理财建议系统
- 掌握纯前端单页应用开发技术

2. 实验环境

检测项	当前状态
Python	已安装 (版本 3.13.1)
Git	已安装
TRAE IDE	国内版, 已登录
ECharts CDN	在线加载

3. 核心设计

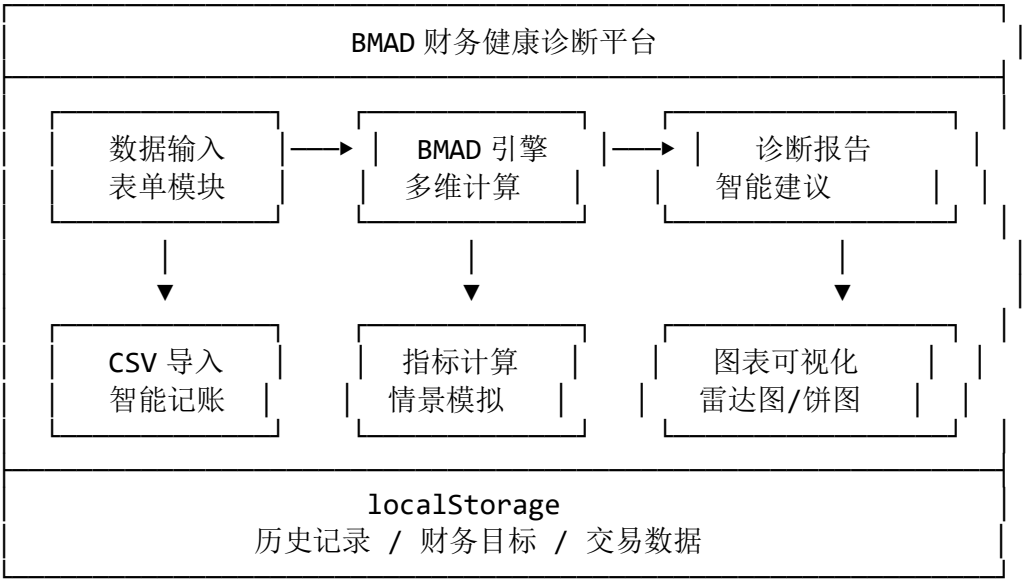
3.1 BMAD 财务健康评估模型

本平台采用四维评估模型 (Balance 收支平衡、Management 负债管理、Accumulation 财富积累、Defense 风险防御):

维度	评估指标	计算公式	健康标准
Balance	储蓄率	(月收入-月支出)/月收入	≥20%

维度	评估指标	计算公式	健康标准
Management	收支比	月支出/月收入	≤80%
	债务收入比	月还款额/月收入	≤36%
	资产负债率	总负债/总资产	≤50%
Accumulation	应急储备月数	流动资产/月支出	≥3-6 个月
	净资产增长率	(期末净资产-期初净资产)/期初净资产	≥5%
Defense	保险覆盖率	已购保额/所需保额	≥80%
	投资分散度	单一资产占比	≤30%

3.2 系统架构设计



4. 功能模块详解

4.1 核心诊断模块

功能：用户输入财务数据，系统自动评估财务健康状况

输入字段：

类别	字段	说明
收入	月收入	基本工资+奖金
	其他收入	投资收益、兼职等
支出	固定支出	房租、房贷、社保等
	可变支出	餐饮、购物、娱乐等
	保险支出	各类保险保费
资产	流动资产	现金、存款、货币基金

类别	字段	说明
	投资资产	股票、基金、债券等
	固定资产	房产、车辆等
负债	短期负债	信用卡、短期贷款
	长期负债	房贷、车贷等

输出结果:

- BMAD 四维健康评分（0-100 分）
- 综合评级（优秀/良好/一般/需改善）
- 雷达图可视化展示
- AI 生成的诊断报告和改善建议

4.2 税务优化顾问

功能：个人所得税计算与专项附加扣除优化

支持的专项附加扣除：

项目	扣除标准	说明
子女教育	¥12,000/年/子女	3 岁至博士阶段
房贷利息	¥12,000/年	首套住房贷款
住房租金	¥18,000/年	无租房
赡养老人	¥24,000/年	父母≥60 岁
继续教育	¥3,600/年	学历/职业资格
大病医疗	≤¥80,000/年	医保报销后自付部分

节税建议： - 充分利用各项专项附加扣除 - 个人养老金账户规划 - 合理收入结构优化

4.3 预算管理中心

功能：基于 50/30/20 法则的预算分配与执行追踪

预算法则：

类别	占比	说明
必要支出	50%	房租、餐饮、交通等必需开支
欲望支出	30%	购物、娱乐、旅游等非必需开支
储蓄投资	20%	储蓄、投资、还债

功能特性： - 月度预算设置 - 消费类别进度条展示 - 超支预警提示 - 实际支出对比分析

4.4 保险保障分析

功能： 保障缺口计算与保额建议

保障需求计算：

险种	计算公式	建议保额
寿险	年收入×10 + 总负债	¥100 万-¥500 万
重疾险	年收入×3 或 ¥50 万起	¥50 万-¥100 万
失能险	年收入×5	覆盖 5 年收入损失

配置优先级： 1. 基础医疗保障 2. 重大疾病保险 3. 寿险（家庭经济支柱） 4. 意外保险 5. 失能收入保障

4.5 现金流预测

功能： 未来 12 个月收支趋势预测

预测模型：

月现金流 = 月收入 - 月支出 - 债务还款
期末余额 = 期初余额 + 月现金流

可视化展示： - 月度现金流柱状图 - 累计余额趋势线 - 现金缺口预警标记

4.6 投资组合深度分析

功能： 资产配置分析与定投计算器

资产配置建议：

风险偏好	股票	债券	货币基金
保守型	20%	50%	30%
稳健型	40%	40%	20%
积极型	60%	30%	10%
激进型	80%	15%	5%

定投计算器： - 定期定额投资模拟 - 复利计算展示 - 收益预测图表

4.7 财务日历

功能： 月度财务事项管理

功能特性： - 日历视图展示 - 工资发放日标记 - 账单到期日提醒 - 定投扣款日记录

4.8 知识中心

功能：理财知识文章展示

精选内容：

文章标题	内容概要
财务自由之路	FIRE 理念与实践方法
资产配置入门	如何构建稳健投资组合
定投策略详解	定期定额投资技巧
购房决策指南	买房还是租房？
保险配置攻略	家庭保障规划
预算管理技巧	50/30/20 法则应用

5. 关键代码实现

5.1 BMAD 评估引擎

```
class BMADEngine {
    static calculate(data) {
        const { monthIncome, otherIncome, fixedExpense, variableExpense,
                liquidAssets, investAssets, fixedAssets, shortDebt, longDebt } = data;

        const totalIncome = monthIncome + otherIncome;
        const totalExpense = fixedExpense + variableExpense;
        const totalAssets = liquidAssets + investAssets + fixedAssets;
        const totalDebt = shortDebt + longDebt;
        const netAssets = totalAssets - totalDebt;

        // Balance 维度（收支平衡）
        const savingsRate = totalIncome > 0 ? (totalIncome - totalExpense) / totalIncome : 0;
        const balanceScore = Math.min(100, savingsRate * 500);

        // Management 维度（负债管理）
        const debtRatio = totalAssets > 0 ? totalDebt / totalAssets : 1;
        const managementScore = Math.max(0, 100 - debtRatio * 100);

        // Accumulation 维度（财富积累）
        const emergencyMonths = totalExpense > 0 ? liquidAssets / totalExpense : 0;
        const accumulationScore = Math.min(100, emergencyMonths * 20);

        // Defense 维度（风险防御）
        const insuranceCoverage = data.insuranceExpense > 0 ? 80 : 40;
```

```

    const defenseScore = insuranceCoverage;

    const avgScore = (balanceScore + managementScore + accumulationScore + defenseScore) / 4;

    return {
        balanceScore, managementScore, accumulationScore, defenseScore,
        avgScore, savingsRate, debtRatio, emergencyMonths,
        totalIncome, totalExpense, totalAssets, totalDebt, netAssets,
        liquidAssets, investAssets, fixedAssets, shortDebt, longDebt
    };
}

```

5.2 ECharts 雷达图配置

```

function renderRadar(results) {
    const chart = echarts.init(document.getElementById('radarChart'));
    const option = {
        radar: {
            indicator: [
                { name: '收支平衡', max: 100 },
                { name: '负债管理', max: 100 },
                { name: '财富积累', max: 100 },
                { name: '风险防御', max: 100 }
            ],
            radius: '60%'
        },
        series: [{
            type: 'radar',
            data: [{
                value: [
                    results.balanceScore,
                    results.managementScore,
                    results.accumulationScore,
                    results.defenseScore
                ],
                name: '财务健康评分'
            }]
        }]
    };
    chart.setOption(option);
}

```

5.3 资产负债结构饼图

```

function renderAssetDebtChart(results) {
    const chart = echarts.init(document.getElementById('assetDebtChart

```

```

    '));
    const data = [
      { value: results.liquidAssets, name: '流动资产' },
      { value: results.investAssets, name: '投资资产' },
      { value: results.fixedAssets, name: '固定资产' },
      { value: results.shortDebt, name: '短期负债' },
      { value: results.longDebt, name: '长期负债' }
    ].filter(item => item.value > 0);

    chart.setOption({
      tooltip: { trigger: 'item' },
      legend: { orient: 'horizontal', bottom: 0 },
      series: [{
        type: 'pie',
        radius: '50%',
        data: data
      }]
    });
  }
}

```

6. 测试验证

6.1 测试数据

项目	数值
月收入	¥15,000
其他收入	¥2,000
固定支出	¥5,000
可变支出	¥3,000
流动资产	¥80,000
投资资产	¥50,000
固定资产	¥200,000
短期负债	¥10,000
长期负债	¥150,000

6.2 预期结果

指标	计算结果
储蓄率	52.9%
资产负债率	48.5%
应急储备月数	10 个月
综合评分	优秀（85 分以上）

7. 问题与解决

问题	解决方案
资产负债饼图数据缺失	在 results 对象中补充 liquidAssets、investAssets、fixedAssets、shortDebt、longDebt 属性
图表容器为空引用	添加容器存在性检查和空数据处理
数据类型错误导致计算异常	添加输入验证和默认值处理
本地服务器停止导致页面无法访问	重新启动 Python HTTP Server

核心代码:

// 资产负债数据补充

```
return {
  ...d,
  balanceScore, managementScore, accumulationScore, defenseScore,
  avgScore, savingsRate, debtRatio, emergencyMonths,
  totalIncome, totalExpense, totalAssets, totalDebt, netAssets,
  liquidAssets: d.liquidAssets,
  investAssets: d.investAssets,
  fixedAssets: d.fixedAssets,
  shortDebt: d.shortDebt,
  longDebt: d.longDebt
};
```

// 图表容器检查

```
const container = document.getElementById('chartContainer');
if (!container) return;
const chart = echarts.init(container);
```

// 输入验证

```
const income = Number(document.getElementById('monthIncome').value) || 0;
```

8. 实验心得

本次实验让我深入理解了 BMAD 方法论在金融科技产品开发中的应用。通过 AI 辅助，我快速完成了一个功能完整的财务健康诊断平台，涵盖了税务优化、预算管理、保险分析、投资组合等多个核心模块。

在开发过程中，我发现 AI 生成的代码需要仔细验证——例如资产负债图表的数据传递问题，以及图表渲染的边界条件处理。这让我意识到，AI 是强大的辅助工具，但人工审核和测试仍然是确保代码质量的关键环节。

同时，我也体会到了纯前端应用的优势：无需后端服务器，部署简单，用户数据存储在本地，隐私安全性高。这种架构非常适合个人财务管理工具这类轻量级应用。

实验总结

通过本次课程实验，我完成了以下工作：

- 1. **实验一：**搭建 TRAE 开发环境，使用 Builder 模式生成贪吃蛇游戏，体验 AI 协作闭环，并完成游戏市场前景分析报告
- 2. **实验二：**创建自定义 Skill（bank-risk-alert），集成 Stata MCP 和 OpenBB Finance 工具，实现 PPT 自动化生成
- 3. **实验三：**使用 BMAD 方法论开发银行 CRM 全栈系统（React+Express+MongoDB）
- 4. **实验四：**开发 BMAD 财务健康诊断平台（HTML+CSS+JavaScript+ECharts），实现多维财务评估、税务优化、预算管理、保险分析、现金流预测、投资组合分析等功能

核心收获：

- AI 是辅助工具，不能完全依赖，所有产出需要人工复核
- MCP 给 AI”手脚”（调用外部工具），Skill 给 AI”专业方法”（领域知识），两者协同才能真正赋能金融业务
- BMAD 方法论建立了规范化开发流程，提高代码质量和开发效率
- 规范化的文档和代码管理（CNB 托管）是项目开发的必要保障
- 纯前端应用适合轻量级金融工具，部署简单且用户隐私安全

附录：CNB 仓库文件清单

核心代码文件

文件路径	对应实验	说明
snake_game.html	实验一	贪吃蛇游戏
snake_market_analysis.md	实验一	游戏市场前景分析报告
SKILL_bank-risk-alert.md	实验二	银行风险预警自定义 Skill
stata-mcp-config.json	实验二	Stata MCP 配置
test_stata.do	实验二	Stata 测试脚本
openbb_demo.py	实验二	OpenBB 金融分析演示
openbb_demo_output.txt	实验二	OpenBB 演示输出
create_ppt.py	实验二	PPT 创建脚本
create_full_ppt.py	实验二	完整 PPT 生成

文件路径	对应实验	说明
pritzker_2026.pptx	实验二	生成的 PPT 文件

前端项目 (frontend/)

文件路径	说明
frontend/src/App.tsx	React 应用入口
frontend/src/pages/*.tsx	页面组件 (Login、Dashboard、CustomerList 等)
frontend/src/components/Layout.tsx	布局组件
frontend/src/services/*.ts	API 服务层
frontend/src/store/authStore.ts	Zustand 状态管理
frontend/package.json	前端依赖配置
frontend/tsconfig.json	TypeScript 主配置
frontend/tsconfig.app.json	应用 TypeScript 配置
frontend/tsconfig.node.json	Node 类型配置
frontend/vite.config.ts	Vite 构建配置
frontend/eslint.config.js	ESLint 代码检查配置

后端项目 (backend/)

文件路径	说明
backend/server.js	Express 服务器入口
backend/controllers/*.js	业务控制器
backend/models/*.js	Mongoose 数据模型
backend/routes/*.js	API 路由定义
backend/middleware/auth.js	JWT 认证中间件
backend/package.json	后端依赖配置

产品需求文档 (docs/)

文件路径	说明
docs/bank-crm-prd.md	银行 CRM 产品需求文档
docs/PRD.md	通用 PRD 模板
docs/TEST_REPORT.md	测试报告

部署文档

文件路径	说明
DEPLOYMENT_GUIDE.md	Gitee Pages 部署指南
README_CN.md	项目中文说明文档

BMAD 方法论配置 (_bmad/)

文件路径	说明
_bmad/config.toml	BMAD 主配置文件
_bmad/agents.md	AI 代理配置
_bmad/workflow.md	开发工作流程
_bmad/_config/version.toml	版本配置
_bmad/_config/defaults.toml	默认配置
_bmad/_config/README.md	内部配置说明
_bmad/custom/custom.toml	用户自定义配置
_bmad/custom/README.md	自定义配置说明
_bmad/bmm/config.toml	BMM 模块配置
_bmad/bmm/templates.toml	BMM 模板配置
_bmad/core/init.ps1	项目初始化脚本
_bmad/core/README.md	核心脚本说明
_bmad/scripts/validate.ps1	配置验证脚本
_bmad/scripts/README.md	脚本说明

Python 自动化脚本

文件路径	功能
generate_loan_pricing.py	贷款 FTP 定价模型
generate_report.py	报告自动生成
generate_ppt_from_excel.py	Excel 转 PPT
update_ppt_with_template.py	PPT 模板更新
openbb_demo.py	OpenBB 金融数据分析
openbb_demo_simple.py	OpenBB 简化演示

旅行攻略（已整合至实验一拓展二）

文件路径	说明
trip_guide.html	旅行攻略 HTML（实验一拓展）

文件路径	说明
travel_guide.json	旅行指南数据（实验一拓展）
travel_map.html	旅行地图页面（实验一拓展）

BMAD 财务健康诊断平台（实验四）

文件路径	说明
bmad-finance.html	财务健康诊断平台主页面（包含所有功能模块）
test-transactions.csv	测试交易数据（用于 CSV 导入功能测试）

平台功能模块清单：

模块	功能说明
核心诊断	BMAD 四维评估、雷达图、饼图、桑基图、AI 智能建议
税务优化顾问	个人所得税计算、专项附加扣除、节税建议
预算管理中心	50/30/20 法则、预算分配、执行追踪、超支预警
保险保障分析	寿险/重疾/失能保障缺口计算、保额建议
现金流预测	12 个月收支趋势、余额预测、缺口预警
投资组合分析	资产配置、收益风险评级、定投计算器
财务日历	月度视图、工资/账单/定投事项管理
知识中心	理财知识文章展示

附录：完整配置文件内容

A.1 前端 TypeScript 配置：frontend/tsconfig.app.json

配置项	值	说明
target	ES2020	目标 ES 版本
lib	ES2020, DOM, DOM.Iterable	包含的标准库
module	ESNext	模块系统
jsx	react-jsx	JSX 转换模式
strict	true	严格类型检查
noEmit	true	不生成 JS 文件（由 Vite 处理）

A.2 Vite 构建配置：frontend/vite.config.ts

配置项	值	说明
plugins	[react()]	启用 React 插件

A.3 前端核心依赖

依赖	版本	用途
react	^18.2.0	前端框架
react-router-dom	^6.22.0	路由管理
antd	^5.15.0	UI 组件库
zustand	^4.5.2	状态管理
echarts	^5.5.0	数据可视化
axios	^1.6.7	HTTP 请求

A.4 后端核心依赖

依赖	版本	用途
express	^4.18.2	后端框架
mongoose	^7.5.0	MongoDB 驱动
cors	^2.8.5	跨域处理
jsonwebtoken	^9.0.2	JWT 认证
bcryptjs	^2.4.3	密码加密
dotenv	^16.3.1	环境变量

A.5 部署指南

部署方式：Gitee Pages 部署

步骤：登录 Gitee → 仓库设置 → Gitee Pages → 选择 gh-pages 分支 → 启动

访问地址：<https://cnb.cool/plus0914/xixixixi>

A.6 产品需求文档核心功能

模块	功能需求
客户管理	列表展示、详情查看、新增/编辑/删除
产品推荐	规则配置、智能推荐、推荐理由展示
报表中心	客户分析、业绩统计、原因分析
系统管理	用户登录认证、权限管理

访问说明

- **银行 CRM 系统：**<https://cnb.cool/plus0914/xixixixi>
- **默认登录：**用户名 admin，密码 admin123